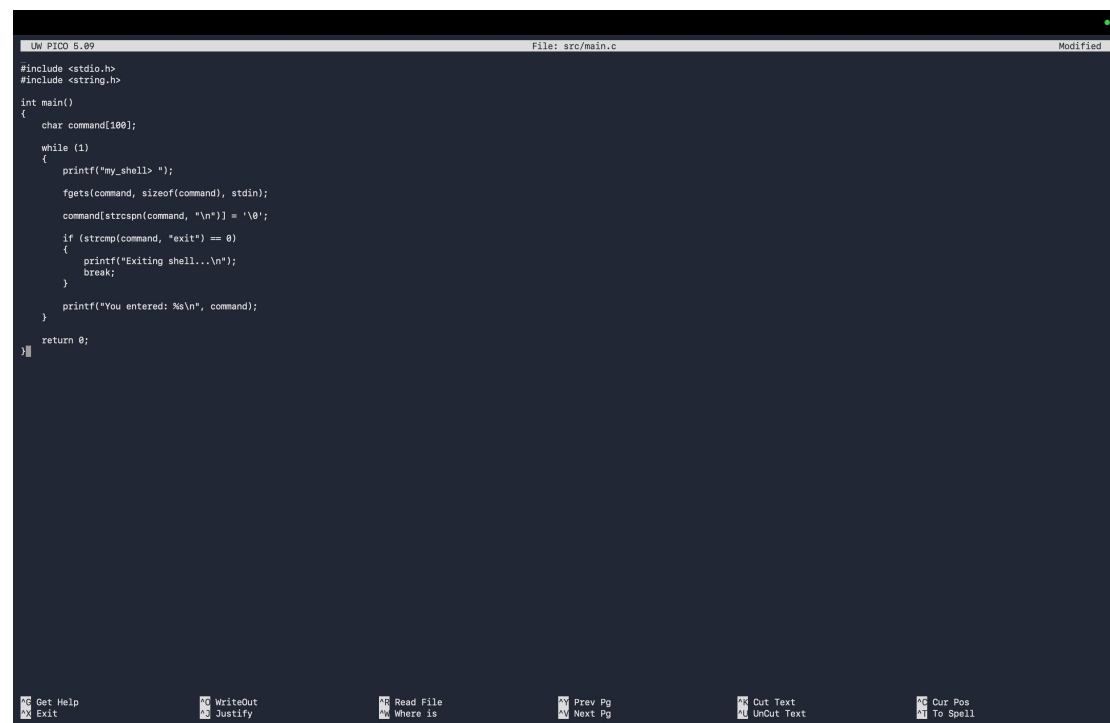


SKILL_WEEK-3

Task 1:

```
root@Vishnu:~# cd ~/OSSP
root@Vishnu:~# pwd
/Users/Vishnu/OSSP
root@Vishnu:~#
```

Figure 1: OSSP project directory

A screenshot of a code editor window titled "UM PIDO 5.09" showing a C program. The file path is "File: src/main.c" and it is marked as "Modified". The code implements a simple interactive shell loop. It includes <stdio.h> and <string.h>. In the main function, it declares a char array 'command' of size 100. It enters a while loop that prints "my_shell" and prompts the user with 'fgets'. It then checks if the command is empty or contains '\0'. If it's empty, it prints "Exiting shell...\n" and breaks. Otherwise, it prints "You entered: \n" followed by the command. The loop continues until the user enters 'exit'.

```
UM PIDO 5.09 File: src/main.c Modified
#include <stdio.h>
#include <string.h>

int main()
{
    char command[100];
    while (1)
    {
        printf("my_shell ");
        fgets(command, sizeof(command), stdin);
        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
        {
            printf("Exiting shell...\n");
            break;
        }

        printf("You entered: %s\n", command);
    }
    return 0;
}
```

Figure 2: C program implementing the interactive shell main loop

```
root@Vishnu:~# gcc src/main.c -o bin/my_shell
root@Vishnu:~#
```

Figure 3: Successful compilation of the shell program

```

root@Vishnu:~# ./bin/my_shell
my_shell> hello
You entered: hello
my_shell> test
You entered: test
my_shell> █

```

Figure 4: Interactive shell prompt accepting user input

```

my_shell> exit
Exiting shell
root@Vishnu:~# █

```

Figure 5: Shell exit condition

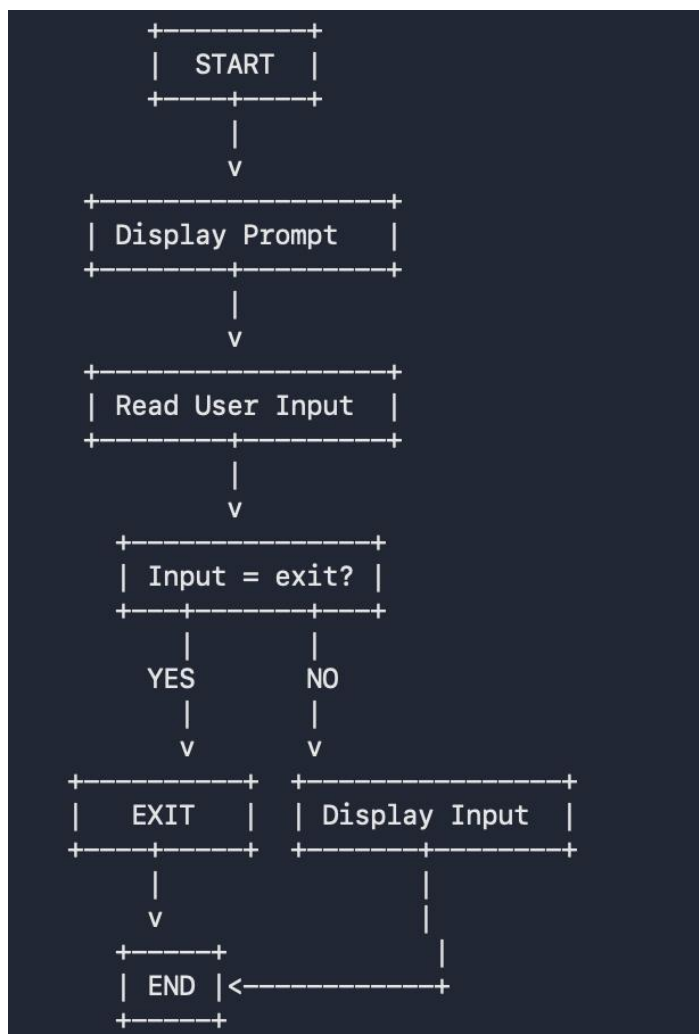


Figure 6: Control flow diagram of the interactive shell

```

root@Vishnu:~# git status
On branch main
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   bin/my_shell
        deleted:    hello
        deleted:    hello.c
        modified:   src/main.c
        deleted:    src/main.c.save

```

Figure 7: Git status showing changes made to the shell main loop

```

delete mode 100644 src/main.c.save
On branch main
nothing to commit, working tree clean
root@Vishnu:~#

```

Figure 8: Final Git status showing a clean working tree.

Task 2:

```

root@Vishnu:~# cd ~/OSSP
root@Vishnu:~# pwd
/Users/Vishnu/OSSP
root@Vishnu:~#

```

Figure 1: OSSP project directory

```

root@Vishnu:~# touch File1.txt
root@Vishnu:~# echo "This is the content of File1." > File1.txt
root@Vishnu:~# cat File1.txt
This is the content of File1.
root@Vishnu:~#

```

Figure 2: Content of File1.txt before copying

```
UW PIDO 5.09 File: src/file_copy.c Modified
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int fd1, fd2;
    char buffer[100];
    int n;

    fd1 = open("File1.txt", O_RDONLY);

    if (fd1 < 0)
    {
        printf("Error opening File1.txt\n");
        return 1;
    }

    fd2 = open("File2.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);

    if (fd2 < 0)
    {
        printf("Error opening File2.txt\n");
        close(fd1);
        return 1;
    }

    n = read(fd1, buffer, sizeof(buffer));

    if (n < 0)
    {
        printf("Error reading File1.txt\n");
        close(fd1);
        close(fd2);
        return 1;
    }

    write(fd2, buffer, n);

    close(fd1);
    close(fd2);

    printf("File copied successfully.\n");

    return 0;
}
```

Figure 3: C program using open(), read(), write(), and close() system calls

```
root@Vishnu:~# nano src/file_copy.c
root@Vishnu:~# gcc src/file_copy.c -o bin/file_copy
root@Vishnu:~# ./bin/file_copy
File copied successfully.
root@Vishnu:~#
```

Figure 4: Successful compilation and execution of the file-copy program

```
root@Vishnu:~# OSSP % cat File1.txt
This is the content of File1.
root@Vishnu:~# OSSP % cat File2.txt
This is the content of File1.
root@Vishnu:~# OSSP % ls -l File1.txt File2.txt
-rw-r--r-- 1 Vishnu staff 30 Aug 13 19:58 File1.txt
-rw-r--r-- 1 Vishnu staff 30 Aug 13 20:04 File2.txt
root@Vishnu:~# OSSP %
```

Figure 5: Verification of copied content in File2.txt and File1.txt and File2.txt after the copy operation

```

root@Vishnu:~# git status
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        File1.txt
        File2.txt
        bin/file_copy
        src/file_copy.c

nothing added to commit but untracked files present (use "git add" to track)
root@Vishnu:~# git add File1.txt File2.txt src/file_copy.c bin/file_copy
root@Vishnu:~# git commit -m "Implement file copy using system calls"
[main a319146] Implement file copy using system calls
root@Vishnu:~#
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly:

    git config --global user.name "Your Name"
    git config --global user.email you@example.com

After doing this, you may fix the identity used for this commit with:

    git commit --amend --reset-author

4 files changed, 48 insertions(+)
create mode 100644 File1.txt
create mode 100644 File2.txt
create mode 100755 bin/file_copy
create mode 100644 src/file_copy.c
root@Vishnu:~# git status
On branch main
nothing to commit, working tree clean
root@Vishnu:~#

```

Figure 6: Git status showing files created for Task2 and clean Git status after completing Task 2